BioHackathon series:
EuroBioC26 Hackathon
Turku, Finland, 2026
Project 2

Submitted: 02 Jun 2026

Published by BioHackrXiv.org

BiocExecute: Make package functions or workflows executable in the command line

Guillaume Deflandre [1], **Leopold Guyot** [2], **Rasmus Hindstrom** [3], **and Claire Rioualen** [4]

1 First Affiliation **2** Second affiliation **3** Third affiliation **4** IFB-core, French Institute of Bioinformatics (IFB), CNRS, INSERM, INRAE, CEA, 94800 Villejuif, France

Introduction

Commandline executables for bioconductor functions and scripts

This project was originally envisioned as a package or function for converting package maintainer or user supplied scripts into commandline executables. The R and Bioconductor paradigms of interactivity through a responsive and informative REPL have been a formula for success for academic users for a long time. But as computational biology becomes increasingly interdisciplinary and increasingly relies on high performance compute, high throughput compute and experimentation, and cloud compute, formalizing portable and flexible implementations of R and Bioconductor tooling is an imperative to ensure that that work, and Bioconductor itself maintain relevancy.

There already exist at least two tools that do 'app' style script conversion; R2G2 - Galaxy specifc integration and Rapp - commandline executables. So developing a package for this project may not be necessary. Long term, the goal of this project is to lay the groundwork for programmatic generation of tooling within Bioconductor packages that can be slotted into modern workflow management systems, or other interactive platforms like Galaxy.

Key Considerations:

Where is the right place for 'app-ification' of a script to take place? Should it happen upon package installation? Overhead checking of things like package versions or argument types are mostly cheap, how much of those checks should we enforce? What do graceful failures for these scripts look like?

Motivation

Bioconductor has a collection of more than 2,400 open source software packages downloaded millions of times per year. They are thoroughly maintained and documented, and their quality is ensured through the use of BiocCheck.

This package aims to further improve Bioconductor software FAIRness, by making them usable in the command line.

- Findability: xxx;
- Accessibility: allows more users to use Bioconductor packages, in a wider variety of setups;
- Interoperability: packages can be combined as modules with other command line tools;
- Reusability: modular components can be reused in future workflows with any workflow manager.

Package users and developers can both benefit from this setup: users can use Bioconductor tools out side of R scripts and integrate them in their own workflows, and package developers can benefit from a wider range of users. Overall, Bioconductor software can gain visibility among a larger community of bioinformaticiens.

Light weight, relies on existing packages

Results

BiocExecute

`BiocExecute` is a package to make Bioconductor/R package functions executable in the Command Line Interface (CLI).

The package comes with a few functions that need to be called upon building a package or upon using the package in the CLI.

Usage

How to use executables

Here's a quick example of how you would call the function `name(x, y)` from the package `pkgExample`:

First, the user needs to make sure the package functions are executable:

```
{r inst, eval = FALSE} BiocExecute::installExecs("pkgExample")
```

And now call those functions from within the terminal:

```
{bash ex, eval = FALSE} pkgExample --help pkgExample name -x Bilbo -y Bag
gins
```

How to create executables

`BiocExecute` uses `Rapp` to make your package functions executable. In the coming sections we will show the different ways `Rapp` includes arguments to be called in the CLI. Note that `Rapp` by itself works with scripts in which the first line is defined as `#!/usr/bin/env Rapp`. You should ***NOT*** include this line in your scripts ! This is handled internally as it is bundled in the `BiocExecute` package.

The executables can have many ranges, from a simple function call to an entire complex workflow. Note that bigger workflows mean more parameters to call in the CLI.

As a maintainer, you may choose which functions/workflows you decide to include in your package. As a package user, we suggest to create pull requests on the package GitHub repository for workflows that you believe should be easily accessible. Try and avoid creating strongly personal workflows. Hold priority for workflows that are repeatedly used across different user cases.

Author contributions

Guillaume Deflandre: Author, creator Leopold Guyot: Author Rasmus Hindstrom: Author Claire Rioualen: Author